

Signal Processing, Machine Learning and Control

HW3

Sravanth Chowdary Potluri
und5uv
CS6762

October 24, 2025

1

1.1 Entropy

Entropy is a measure of impurity or uncertainty within a set of examples. In classification, it quantifies how mixed a set of data is in terms of its class labels. A set that is perfectly pure, meaning all examples belong to the same class, has an entropy of 0. Conversely, a set that is evenly divided between classes (e.g., 50% "Yes" and 50% "No") has the maximum possible impurity and an entropy of 1. The general formula for entropy for a set S with c classes is:

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2 p_i$$

where p_i is the proportion of examples in S that belong to class i .

1.2 Information Gain and its Relationship to Entropy

Information Gain is the metric used in decision trees to select the best attribute (or feature) to split the data on at each node. The core relationship between the two concepts is that **Information Gain is the measure of the reduction in entropy**. When a dataset S is split using an attribute A , the Information Gain measures how much the uncertainty of the set was reduced by that split.

The calculation for gain, $Gain(S, A)$, is:

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

This formula subtracts the weighted average entropy of all the child subsets (created by splitting on attribute A) from the original entropy of the parent set S . S_v represents the subset of examples where attribute A has the value v . To build the tree, the algorithm calculates this gain for every possible attribute, and the attribute that provides the **maximum Information Gain** is chosen as the splitting node.

2

To compute the entropy, we use the general formula for a set S with c classes:

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2 p_i$$

where p_i is the proportion of examples belonging to class i .

2.1 Step 1: Determine Proportions

First, we find the total number of balls in the urn:

- Total Balls = 3 (Red) + 2 (Green) + 1 (White) = 6

Next, we calculate the proportion (p_i) for each color:

- $p_{\text{red}} = \frac{3}{6} = 0.5$
- $p_{\text{green}} = \frac{2}{6} \approx 0.333$
- $p_{\text{white}} = \frac{1}{6} \approx 0.167$

2.2 Step 2: Calculate Entropy

Now, we plug these proportions into the entropy formula:

$$Entropy = -[(p_{\text{red}} \cdot \log_2 p_{\text{red}}) + (p_{\text{green}} \cdot \log_2 p_{\text{green}}) + (p_{\text{white}} \cdot \log_2 p_{\text{white}})]$$

$$Entropy = -[(0.5 \cdot \log_2 0.5) + (\frac{2}{6} \cdot \log_2 \frac{2}{6}) + (\frac{1}{6} \cdot \log_2 \frac{1}{6})]$$

$$Entropy = -[(0.5 \cdot -1.0) + (0.333 \cdot -1.585) + (0.167 \cdot -2.585)]$$

$$Entropy = -[-0.5 - 0.528 - 0.432]$$

$$Entropy = -[-1.46]$$

$$Entropy \approx 1.46$$

The entropy of the urn is approximately **1.46**.

3

3.1 What is a Random Forest Classifier?

A Random Forest is an **ensemble classifier** that operates by constructing a large number of individual decision trees during training. This process incorporates two key forms of randomization to ensure the trees are different from one another.

First, each tree is trained on a "bootstrapped" dataset, which is a random sample of the original training data selected with replacement (a technique called **bagging**). Second, when building each tree, the split at each node is determined by searching over a **random subset of features** (attributes) rather than all of them.

To classify a new instance, that instance is passed down every tree in the forest. Each tree "votes" for a class, and the final classification is determined by the **majority vote** from all the trees.

3.2 Advantages Over a Decision Tree

The primary advantage of a Random Forest over a single Decision Tree is its superior accuracy and robustness against **overfitting**. A single decision tree is highly prone to overfitting; it can learn the noise and specific quirks of the training data too well, causing it to perform poorly on new, unseen data.

By averaging the results of many different trees (which are "decorrelated" by the random data and feature sampling), the Random Forest is able to generalize much better to new data, leading to higher predictive accuracy. Because of this ensemble approach, Random Forests also do not require **pruning**, a process that is often necessary to reduce the complexity and prevent overfitting in a single decision tree.

4

4.1 Sigmoid Function

The **sigmoid function** is an activation function that takes any real-valued input and "squashes" it into a range between 0 and 1. This S-shaped curve is useful for models that need to predict a probability as an output.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

4.2 Tanh Function

The **tanh function**, or hyperbolic tangent, is another activation function that is similar in shape to the sigmoid. However, it squashes its real-valued input into the range of -1 to 1. It is defined as:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\tanh(x) = 2\sigma(2x) - 1$$

4.3 Softmax Function

The **softmax function** is a normalized exponential function. It is commonly used in the output layer of a multi-class classifier because it takes a vector of arbitrary real-valued scores (logits) and converts them into a vector of values between 0 and 1 that sum to 1. This output can be interpreted as a probability distribution over the different classes. The formula for the softmax of an element y_i in a vector is:

$$S(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$

5

Training a neural network begins with initializing its weights and biases, often with small random values. The network is then fed batches of training data. For each input, a **forward propagation** step occurs: the input data is passed through the network's layers, with each neuron performing a weighted sum of its inputs followed by an activation function (like sigmoid, tanh, or ReLU). This process continues until the final layer produces an output, or prediction.

After the forward pass, a **loss function** (e.g., mean squared error) is used to calculate the "error," which is the difference between the network's predicted output and the true label from the training data. This error is then used in **backpropagation**, an algorithm that propagates the error backward through the network to calculate the gradient (the partial derivative of the loss) with respect to each weight and bias. These gradients indicate how much each weight and bias contributed to the total error. Finally, an optimization algorithm, such as **Gradient Descent**, updates the weights and biases in the direction that minimizes the loss. This entire cycle of forward pass, loss calculation, backpropagation, and weight update is repeated many times (for many epochs) until the network's predictions are sufficiently accurate.

6

Two applications where a Deep Neural Network (DNN) can be used are:

- **Smart Health:** A DNN can be used to analyze features extracted from a person's acoustic signal (voice) to help determine their mental state, such as their mood, stress, or anxiety levels.
- **Autonomous Vehicles:** A DNN, particularly a Convolutional Neural Network (CNN), can be used to process pixel data from a camera to detect objects in a scene. A specific example is an autonomous vehicle seeing and identifying a stop sign.

7

Although Convolutional Neural Networks (CNNs) are famous for image processing, they are fundamentally pattern-recognition tools that can be applied to any data that can be structured like an image. This makes them highly effective for time-series data, which can be formatted into 2D matrices where one axis is time and the other is a measured feature.

7.1 Audio and Speech Recognition

A CNN can be used for tasks like speech recognition or classifying sounds.

Input: The raw input is an audio clip, which is a 1D time-series signal. To use a CNN, this signal is pre-processed. It is first broken into short, overlapping time windows (e.g., 25ms windows sliding by 10ms).

Processing: A Discrete Fourier Transform (DFT) is applied to each window to calculate the frequency components, creating a **spectrogram**. This spectrogram is a 2D

matrix where one axis represents the time windows and the other axis represents frequency bins (showing the intensity of each frequency at each moment). This 2D representation is treated by the CNN as an "image."

Output: The CNN's convolutional filters can then learn to detect patterns (specific shapes) in this spectrogram that correspond to different sounds. The eventual output is a classification, such as identifying a spoken word, the mood of the speaker, or detecting a specific sound like a whale call.

7.2 Activity Recognition from Sensor Data

A CNN can be used to classify a user's physical activity using sensor data from a wearable device.

Input: The input is time-series data from a multi-axis sensor, such as a 3D accelerometer (with X, Y, and Z axes) on a smartphone or smartwatch.

Processing: To prepare this data for a CNN, the readings are collected over a fixed time window (e.g., 1 second of data, sampled 60 times). This creates a 2D matrix where the rows correspond to the sensor axes (X, Y, Z) and the columns represent the time steps (60 samples). This 3×60 matrix is treated as a small "picture."

Output: The CNN's filters slide across this matrix, learning spatio-temporal patterns unique to different activities. The eventual output is a classification label for that 1-second window, such as "walking," "running," or "jumping."

8

An LSTM (Long Short-Term Memory) cell uses a series of "gates" to regulate the flow of information, allowing it to maintain a long-term memory (the cell state). These gates are essentially small neural networks that learn which information is important to add, remove, or output. The three main gates are the Forget Gate, the Input Gate, and the Output Gate.

All three gates share the same inputs: the **previous hidden state** (h_{t-1}) and the **current input** (x_t). These two vectors are concatenated and fed into each gate.

8.1 Forget Gate

- **Purpose:** The forget gate's job is to decide what information to throw away from the previous cell state (C_{t-1}).
- **Inside the Gate:** It consists of a single **sigmoid** (σ) layer.
- **Output:** It outputs a vector f_t with values between 0 and 1 for each number in the cell state. A '1' means "completely keep this" information, while a '0' means "completely get rid of this".

8.2 Input Gate

- **Purpose:** The input gate decides which new information will be stored in the cell state. This is a two-part process.
- **Inside the Gate:** It has two layers:

1. A **sigmoid (σ) layer** that decides which values to update.
 2. A **tanh layer** that creates a vector of new candidate values, $\tilde{C}_t$, that could be added to the state.
- **Output:** The gate outputs two vectors. The first (from the sigmoid layer) is a 0-to-1 filter, i_t , that scales the candidate values. The second (from the tanh layer) is the candidate vector $\tilde{C}_t$ itself. These are then combined and added to the cell state.

8.3 Output Gate

- **Purpose:** This gate determines what the next hidden state (h_t) will be. The hidden state is a filtered version of the cell state that is passed on to the next time step and used for predictions.
- **Inside the Gate:** It also consists of two parts:
 1. A **sigmoid (σ) layer** that decides which parts of the cell state to output.
 2. The current cell state (C_t) is passed through a **tanh layer** to squash its values between -1 and 1.
- **Output:** The final output of the LSTM cell, the hidden state h_t , is calculated by multiplying the output of the sigmoid gate (o_t) with the output of the tanh-processed cell state ($\tanh(C_t)$).

9

The key issues in developing Deep Neural Networks (DNNs) for execution on small devices are the significant resources these models demand. Small, embedded, and mobile devices, such as microcontrollers, smartwatches, or IoT edge devices, are highly constrained.

The main challenges are:

- **Memory:** DNNs require a significant amount of memory to store the network's weights and intermediate activations. Small devices often lack the memory capacity to hold these large models.
- **Processing Speed:** These models are computationally intensive and require high processing speed. Small devices with less powerful processors may not be able to run the network's inference "fast enough" for real-time applications.
- **Energy Consumption:** The high processing load leads to significant energy consumption. This is a critical problem for devices that are battery-powered, such as smart phones or watches.

10

10.1 ML Applications in Autonomous Systems

Machine Learning (ML) is a critical component of autonomous systems, such as autonomous vehicles. It is used to interpret complex sensor data and make real-time deci-

sions. For instance, a Deep Neural Network (DNN), particularly a Convolutional Neural Network (CNN), can process pixel data from a camera. This allows the system to perceive its environment, such as an autonomous vehicle "seeing" and identifying a stop sign.

Beyond perception, ML models like Decision Trees can be used for high-level decision-making. This includes deciding what action the vehicle should take (e.g., stop, slow down) or determining the optimal route to follow.

10.2 Challenges

A primary challenge is that the sophisticated ML models required for these tasks (like DNNs) demand a significant amount of resources. Autonomous systems often rely on small, embedded devices which have major constraints. These key challenges include:

- **Memory:** DNNs require a large amount of memory to store their weights and biases, which small devices often lack.
- **Processing Speed:** These models are computationally intensive. The limited processing speed of small devices can make it difficult to run the model's inference fast enough for real-time control.
- **Energy:** The high processing load leads to high energy consumption, which is a critical problem for battery-powered systems.

11

11.1 ML Applications in Smart Health

Machine Learning (ML) can be used in Smart Health Cyber-Physical Systems (CPS) to analyze sensor data and make real-time decisions that can be acted upon. For example, ML models can analyze a person's features, such as their age, weight, blood pressure, and heart rate, to make decisions. Specific applications include:

- **Decision Support:** A model can decide if a drug should be prescribed or if a person should have an operation.
- **Automated Actuation:** An ML system can determine how much insulin to inject and then actuate the injection.
- **User Feedback:** The system can determine the best advice to give a person and deliver it to the user.
- **Diagnostics:** A model can classify a medical image, such as determining if it shows cancer, or analyze features from an acoustic signal to detect a person's mood, stress, or anxiety.

11.2 Challenges

A primary challenge is that the sophisticated ML models required for these tasks, particularly Deep Learning (DL) models, must often run on small, resource-constrained devices. These devices are common in CPS and present several key issues:

- **Memory:** DL models require a significant amount of memory to store their weights and biases, which often exceeds the capacity of small devices.
- **Processing Speed:** These models are computationally intensive. The limited processing speed of embedded devices can make it difficult to run the model's inference fast enough for real-time applications.
- **Energy:** The high processing load leads to significant energy consumption, which is a critical problem for battery-powered smart health devices.

References

- [1] *CS 6762: Signal Processing, Machine Learning and Control Class Materials and Lectures*. University of Virginia.
- [2] *Other Internet Sources for basic information, facts and examples*.

Acknowledgment

This Homework makes use of Generative AI models like Gemini and ChatGPT for research to form and refine answers as required for this homework and some of the main sources for the information are cited. The author acknowledges that they have not given or received unauthorized assistance on this homework. The Author also acknowledges that they have understood the course content and lectures for the preparation of these homework solutions.